

AI RESEARCH KNOWLEDGE HUB

Project Report

Evidence-based review of implemented system capabilities, architecture, workflows, testing, and remaining work.

Audience: Institutional stakeholders, assessors, administrators; basis: current source and repository documentation;
status: current implementation, not a completion estimate

Contents

1. EXECUTIVE SUMMARY	5
2. INTRODUCTION	5
2.1 Background	5
2.2 Motivation and institutional context	5
2.3 Target users	5
3. PROBLEM STATEMENT	6
4. PROJECT OBJECTIVES	6
4.1 General Objective	6
4.2 Specific Objectives	6
5. SYSTEM OVERVIEW	6
6. USER ROLES AND FUNCTIONALITIES	7
7. SYSTEM REQUIREMENTS	7
7.1 Hardware Requirements	7
7.2 Software and frameworks	7
7.3 Operating systems and browser	7
8. SYSTEM ARCHITECTURE	8
8.1 Presentation layer	8
8.2 API and application layer	8
8.3 Data layer	8
8.4 Local AI layer and communication	8
8.5 Deployment topology	8
9. SYSTEM DESIGN AND WORKFLOW	8
9.1 User authentication	8
9.2 Document upload	9
9.3 Document processing and metadata	9
9.4 Semantic search	9
9.5 Ask AI / RAG	9
9.6 Document summarization	10
9.7 Source references / citations	10
9.8 Saved research	10
9.9 Administrative monitoring	10
10. DATABASE DESIGN	10

10.1 Database technology and schema creation	10
10.2 Entities and relationships	11
11. API DOCUMENTATION	11
11.1 Authentication (/api/auth)	12
11.2 Documents (/api/documents)	12
11.3 Search, chat, summarization, and dashboard	13
11.4 Administration (/api/admin)	13
11.5 Service endpoints	14
12. AI AND RAG FUNCTIONALITY	14
12.1 Embeddings and vector retrieval	14
12.2 Retrieval-augmented question answering	14
12.3 Local language models and summaries	14
12.4 Source flow	14
13. DOCUMENT PROCESSING	14
13.1 Validation and storage	14
13.2 Extraction and normalization	15
13.3 Chunking, embedding, and persistence	15
14. SECURITY AND ACCESS CONTROL	15
Implemented controls	15
Security limitations and deployment considerations	15
15. ADMINISTRATION AND AUDIT	16
15.1 User administration	16
15.2 Dashboards and monitoring	16
15.3 Audit history, notifications, and PDF reporting	16
16. USER INTERFACE	17
17. TESTING AND VALIDATION	18
17.1 Current validation performed for this report	18
17.2 Existing automated test coverage	18
17.3 Historical validation	18
18. CURRENT PROJECT STATUS	18
COMPLETED / IMPLEMENTED	18
IN PROGRESS	19
REMAINING WORK / NOT VERIFIED	19
19. BOT EXPERT FEEDBACK	19

20. CHALLENGES AND LIMITATIONS

20.1 Document processing and metadata

20.2 Search, citations, and RAG

20.3 Model and deployment resources

20.4 Schema and operations

20.5 Documentation drift

21. WAY FORWARD

21.1 Immediate priorities

21.2 Deployment and longer-term validation

22. CONCLUSION

23. APPENDICES

Appendix E - Installation Guide

Appendix G - Testing Evidence

20

20

20

20

20

21

21

21

21

Prepared: 5 October 2026

Evidence basis: Current repository implementation and project documentation

Implementation status: Functioning application; production deployment and user acceptance are not verified by this report

This report distinguishes implemented behavior from documented intent, historical validation, and work not verified in the current implementation. The three Bank of Tanzania expert-feedback items in Section 19 were supplied in the project-report prompt; no dated feedback record was found in the repository.

1. EXECUTIVE SUMMARY

The AI Research Knowledge Hub is a central-banking research document management application. Its documented purpose is to bring institutional research documents into a repository where authorized users can upload and review documents, retrieve content through search, ask questions grounded in indexed documents, and request structured document summaries.

The current implementation comprises a React and Vite frontend, a FastAPI backend, PostgreSQL persistence through SQLAlchemy, local document-file storage, persistent embedded ChromaDB vector storage, SentenceTransformers embeddings, and a local Ollama service for question answering and summaries. Authentication uses bearer JWTs and bcrypt password hashes. The implemented application includes document upload and processing, metadata review and editing, semantic search with a database-backed fallback, RAG responses with source records, summarization, saved research, administrative user management, activity review, dashboard statistics, and PDF activity reporting.

The current backend test suite was run for this report: 26 tests passed. The Vite production build and frontend lint also succeeded after removing stale dashboard code. Docker Compose, live PostgreSQL/Ollama integration, and end-to-end browser acceptance were not tested as part of this report. A separate audit report records historical validation from 5 September 2026; those results are not presented here as current test results.

Overall, the repository contains a working and testable application with substantial document and AI-assisted research workflows. The repository does not establish a formal project completion percentage, institutional acceptance, measured AI accuracy, production readiness, or performance benchmarks. Production deployment hardening, broader integration testing, complete post-upload metadata editing, and consistent page-level citations remain areas for validation or improvement.

2. INTRODUCTION

2.1 Background

The project documentation describes the system as a central-banking research knowledge management application. The README summarizes its scope as document management, semantic search, grounded RAG, and document summarization. The handover document identifies institutional research documents such as economic research, policy reports, and working papers as the intended knowledge base.

2.2 Motivation and institutional context

Research reports and policy documents can be long, distributed across collections, and difficult to retrieve using filenames or exact keywords alone. The implemented system combines a document catalogue, extracted text, semantic retrieval, and AI-assisted workflows to support discovery and review. This describes the solution's intent; the repository does not include a measured baseline of the institution's prior document-search process or quantified time savings.

The interface and project documentation use Bank of Tanzania context and vocabulary. The audit report records that Bank of Tanzania publication areas were used as classification vocabulary. This repository does not verify formal Bank of Tanzania ownership, institutional approval, procurement, deployment, or user acceptance.

2.3 Target users

The backend defines three roles: `viewer`, `researcher`, and `admin`. Viewers can use authenticated research functions; researchers can manage research documents; administrators have separate user, dashboard, and audit controls. These permissions are detailed in Section 6.

3. PROBLEM STATEMENT

The project documentation identifies a need to manage central-banking research documents and make their content discoverable through semantic search, grounded question answering, and summaries. The system addresses the difficulty of locating relevant passages across stored research files and of reviewing long documents by exposing extracted chunks, source metadata, and structured summary output.

The repository does not contain a standalone approved problem-statement document, a quantified estimate of document volume, a formal user-research study, or measured search-time and review-time baselines. Accordingly, those institutional conditions and their scale are **Not implemented / Not verified in the current implementation**.

4. PROJECT OBJECTIVES

4.1 General Objective

A formally approved general objective is **Not verified in the current implementation**: no project specification or objectives document was found in the repository. The documented system purpose is to provide central-banking research document management with semantic discovery, grounded question answering, and summarization. This purpose is reported as documented intent, not as a verbatim approved objective.

4.2 Specific Objectives

A formally approved list of specific objectives is **Not verified in the current implementation**. The code implements the capabilities described in Sections 5 and 9, but implemented features are not substituted here for an approved objectives statement.

5. SYSTEM OVERVIEW

The application provides the following implemented capabilities:

Capability	Current implementation
Research document repository	PostgreSQL document records and local file storage; active documents can be listed and viewed.
Upload and processing	Authenticated researcher/admin upload for PDF and DOCX, file validation, extraction, chunking, metadata assignment, and processing status.
Metadata management	Extraction preview before upload; post-upload editor for title, author, and publication year. Category, document type, and topic editing is not provided by the post-upload editor.
Semantic search	SentenceTransformer query embeddings and ChromaDB retrieval; a PostgreSQL chunk-text fallback is available when semantic service initialization fails or when additional results are needed.
Ask AI / RAG	Retrieves relevant content, builds a context prompt, calls Ollama, and returns an answer with source records when evidence is found.
Summarization	Calls a dedicated Ollama summary model for processed document chunks and returns a structured research brief.
Source references	Search and chat responses carry document/chunk identifiers, excerpts, and optional page number and document metadata. Page-number availability depends on source format and processing data.
Saved research	Per-user saved-document relationship and corresponding save/list/remove endpoints.
User and role management	Administrator-only user creation, listing, role updates, account status updates, and deletion.
Audit and monitoring	Database-backed activity history, notifications, review state, statistics dashboards, and downloadable PDF report.

AI features rely on locally reachable Ollama and the configured models; they are not an autonomous AI service independent of those dependencies. The application has no separate ChromaDB server: it uses ChromaDB's persistent embedded client.

6. USER ROLES AND FUNCTIONALITIES

Role definitions are in `backend/app/models/user.py`. Endpoint authorization is enforced by dependencies in `backend/app/core/security.py` and the relevant routers. Public self-registration assigns the `viewer` role; it does not accept an elevated role from the caller.

Role	Main Capabilities	Restrictions
Viewer	Authenticated document listing/details/file viewing; search; Ask AI; summaries; dashboard summary; own saved research; own profile and password operations.	Cannot upload, edit document metadata, or delete documents. Cannot use admin user-management, administrative dashboard, or audit endpoints.
Researcher	Viewer-facing research functions plus document upload, upload metadata review, post-upload title/author/year editing, and document deletion.	Cannot use admin-only user-management or audit-report endpoints. The frontend restricts administrative sections to admins.
Admin	Research and document-management functions; administrative dashboard; create/list/view/update/delete other user accounts; change other users' role/status; activity monitoring/review; PDF activity export.	Backend prevents an admin from changing their own role/status through those specific user-management operations and prevents deletion of their own account.

The browser frontend and backend both enforce role-oriented access for key flows. The backend remains authoritative: protected routes require an active authenticated account and role dependencies where applicable. Public registration creates a viewer account with `is_verified=False`; the login flow does not perform a separate verification check in the inspected authentication code.

7. SYSTEM REQUIREMENTS

7.1 Hardware Requirements

The repository does not specify minimum CPU, RAM, GPU, disk capacity, or concurrent-user targets. These values are **Not verified in the current implementation**. The actual footprint depends on document volume, Ollama models, embedding model cache, PostgreSQL data, and ChromaDB persistence. No GPU setting is required or declared by the code.

7.2 Software and frameworks

Area	Implemented technology / repository setting
Frontend	React 19 and Vite 8; JavaScript and JSX; npm dependencies are locked in <code>frontend/package-lock.json</code> .
Backend	Python 3.11 container base; FastAPI 0.115.6; Uvicorn 0.34.0; Pydantic 2.10.4.
ORM / migrations	SQLAlchemy 2.0.36; no versioned migration tool or revision files are configured in the current repository.
Relational database	PostgreSQL; Compose uses <code>postgres:16-alpine</code> ; connection uses SQLAlchemy and <code>psycopg2-binary</code> .
Vector storage	ChromaDB 1.5.9 persistent client, stored under repository <code>vectorstore/</code> .
Embeddings	SentenceTransformers 3.3.1 using <code>all-MiniLM-L6-v2</code> ; PyTorch and Transformers are declared dependencies.
Text generation	Local Ollama HTTP API; default RAG/chat model <code>deepseek-r1:1.5b</code> ; default summary model <code>qwen3.5:0.8b</code> .
Document parsing	<code>pypdf</code> , <code>pdfplumber</code> , and <code>python-docx</code> ; accepted upload formats are PDF and DOCX.
Containers	Backend image uses <code>python:3.11-slim</code> ; frontend image uses <code>node:22-alpine</code> for build and <code>nginx:1.27-alpine</code> for static serving.

7.3 Operating systems and browser

Dockerfiles provide Linux-based container execution. The repository README documents a Python 3.11 and Node/npm local workflow and PowerShell examples. A complete operating-system support matrix is **Not verified in the current implementation**.

The frontend is a browser application, but supported browser versions and a browser compatibility test matrix are **Not verified in the current implementation**.

8. SYSTEM ARCHITECTURE

No standalone system architecture diagram or formal system-design document was found in the repository. The following is a code-derived description of the implemented components, not a pre-existing approved architecture diagram.

8.1 Presentation layer

The React/Vite single-page application renders authentication, researcher, viewer, and administrator screens. `frontend/src/services/api.js` creates an Axios client with a default API base URL of `http://localhost:8002/api`. If an `access_token` exists in browser local storage, the request interceptor sends it as a bearer token.

8.2 API and application layer

FastAPI is created in `backend/app/main.py`. It registers authentication, admin, document, search, chat, summarization, and dashboard routers. CORS origins are explicitly listed in that file for local development ports. Swagger UI, ReDoc, and OpenAPI JSON use FastAPI's standard routes (`/docs`, `/redoc`, and `/openapi.json`).

8.3 Data layer

SQLAlchemy connects to PostgreSQL using `DATABASE_URL`. PostgreSQL stores accounts, document metadata, extracted chunks, saved-research links, AI questions, audit records, categories, document types, keywords, topics, and their relationship tables. Original uploaded documents are kept on the backend filesystem under `data/documents/` by default. ChromaDB uses a persistent local directory under `vectorstore/`.

8.4 Local AI layer and communication

The backend loads `all-MiniLM-L6-v2` through SentenceTransformers to embed document chunks and search queries. It queries ChromaDB's `research_documents` collection for vector retrieval. RAG and summarization services call Ollama's `/api/generate` endpoint over HTTP. The backend performs document processing and embedding/indexing synchronously in the upload request; a separate worker queue is not configured.

At a high level, data moves through these actual boundaries:

```
Browser (React/Vite)
-> HTTP/JSON or multipart requests with bearer JWT
FastAPI routers and services
-> PostgreSQL for accounts, document records, chunks, and audit data
-> local filesystem for uploaded source files
-> SentenceTransformers for embeddings
-> persistent ChromaDB for vector search
-> Ollama HTTP API for RAG/chat and document summaries
```

8.5 Deployment topology

The production Compose file declares PostgreSQL, Ollama, backend, and Nginx-served frontend containers. The development Compose file runs Vite as a Node container and publishes PostgreSQL, Ollama, backend, and frontend ports. Compose configuration exists, but Docker Compose execution was not performed for this report.

9. SYSTEM DESIGN AND WORKFLOW

9.1 User authentication

1. A user submits username/email and password to `POST /api/auth/login`.
2. The backend verifies the password against a bcrypt hash, updates `last_login`, writes a login audit activity, and returns an HS256 JWT and user data.
3. The frontend stores the token in local storage and sends it as a bearer token on subsequent Axios requests.
4. Startup session restoration calls `GET /api/auth/me`. Active-account checks are performed on protected endpoints.

Tokens are configured in code to expire after 24 hours. There is no refresh-token workflow in the inspected implementation.

9.2 Document upload

1. A researcher/admin selects a PDF or DOCX in the Upload page.
2. The frontend sends the file to `POST /api/documents/upload/preview` for temporary metadata extraction.
3. The backend checks the extension, reported MIME type, empty content, and 50 MB limit; it extracts metadata using a temporary file that is removed after preview.
4. The user can review title, author, and year before confirming the metadata and upload.
5. The frontend submits multipart data to `POST /api/documents/upload`.
6. The backend checks the file, rejects active duplicate file content using SHA-256, stores the file with a timestamp/UUID-based name under `data/documents/`, creates the database record, processes it, saves chunks, attempts vector indexing, and returns processing details.

9.3 Document processing and metadata

For PDF, the processor extracts page text with `pypdf` and may retry extraction with `pdfplumber` when extracted text is limited. For DOCX, it extracts paragraphs and table content with `python-docx`. The text is cleaned while retaining line boundaries. Metadata rules inspect document text and filename for title, authors, publication year/period, department/category, type, topics, and keywords. This is code-based extraction and classification; the inspected `DocumentProcessor` does not call the LLM to infer metadata.

The processor splits text into sentence-oriented chunks with a configured target of 1,000 characters and 200 characters overlap. PDF chunks retain page numbers. Upload processing stores PostgreSQL chunk rows, marks document status, and attempts embedding generation and Chroma indexing. If embedding indexing fails, the upload response can still report completed text processing and a `processing_error` records the indexing issue; database text search remains available as a fallback.

9.4 Semantic search

1. An authenticated user submits a query to `POST /api/search/`.
2. The backend attempts to initialize the document service, embed the query, and retrieve up to the requested number of relevant vectors from ChromaDB.
3. Results are enriched with current document metadata and page information from PostgreSQL. Inactive or missing documents are filtered.
4. If semantic service initialization fails, the backend uses a lexical database-chunk fallback. When semantic results do not fill the requested result count, fallback results can supplement them.
5. The frontend shows result excerpts and provides a document-view action; when a page number is available, the PDF viewer URL includes a page fragment.

9.5 Ask AI / RAG

1. An authenticated user posts a question and optional `top_k` to `POST /api/chat/`.
2. RAG retrieves relevant chunks through the document service. If semantic service initialization is unavailable, the route uses database chunks and lexical term matching.
3. The RAG service builds a context containing source document and chunk identifiers, retrieved text, and relevance information.
4. The Ollama chat model receives a prompt instructing it to answer from the supplied context and to acknowledge insufficient evidence.
5. On success, the route enriches each source with current document metadata and available page number, persists the question, writes an audit event, and returns the answer and sources.

A no-relevant-content or model failure is represented in the response as `success: false` and an error message. No quantitative answer-accuracy evaluation is included in the repository.

9.6 Document summarization

The authenticated user selects a processed document. The backend reads its ordered PostgreSQL chunks, recovers chunks from the source file if they are absent, combines content, limits the summary evidence window to 18,000 characters, and calls the configured summary model. The service produces a structured institutional research brief and returns document information, summary content, raw summary, and counts. The summary API writes an audit activity after success.

9.7 Source references / citations

Search results and chat source models include `document_id`, `chunk_index`, `excerpt`, and optional `page_number`. For PDFs, page-aware chunking and page numbers are implemented. DOCX extraction provides no page map, so an exact source page is not guaranteed for Word files. Chunk identifiers remain part of the source record and are not a substitute for a page citation.

9.8 Saved research

An authenticated user can save a document, list their own saved documents, and remove a saved relationship. `saved_research` enforces a unique user/document pair. The list route is scoped to the current user's account.

9.9 Administrative monitoring

An admin can inspect user/document/processing statistics, recent activity, role breakdowns, and notifications; change other users' roles and active status; mark activities reviewed; and export a date-filtered PDF report from live database statistics and audit entries. The activity API's history request also marks currently unreviewed rows as reviewed before returning them; notifications have a separate read route.

10. DATABASE DESIGN

10.1 Database technology and schema creation

The application uses PostgreSQL via SQLAlchemy. Database connection is configured through `DATABASE_URL`; the default source value targets a local `knowledge_hub` database. Compose declares PostgreSQL 16. The repository does not define PostgreSQL extensions.

At FastAPI startup, the application invokes SQLAlchemy `Base.metadata.create_all()` and applies limited additive compatibility `ALTER TABLE` statements for older document/audit schemas. `backend/init_db.py` also creates tables and calls seed routines for categories, document types, and keywords. No versioned schema-migration path is configured in the repository.

10.2 Entities and relationships

Entity/Table	Purpose	Important Relationships
users	Login identity, bcrypt password hash, role, active/verified flags, and timestamps.	Referenced by uploaded documents, AI questions, audit logs, and saved-research links.
documents	Document title, author and author metadata, year/period, file location/type/size, processing status, category/type references, uploader, and timestamps.	Optional category/type and uploader; many-to-many topics/keywords; one-to-many chunks; saved-research links.
document_chunks	Extracted chunk text, index, length, Chroma ID, optional page number/section, and document relationship.	Foreign key to documents; document deletion cascades at database definition.
categories	Department/classification names and descriptions.	One-to-many documents.
document_types	Document type names and descriptions.	One-to-many documents.
keywords	Keyword labels used for document tagging.	Many-to-many with documents through document_keyword.
research_topics	Controlled research-topic labels.	Many-to-many with documents through document_topic.
document_keyword	Association table linking documents and keywords.	Foreign keys to documents and keywords; both declared cascading.
document_topic	Association table linking documents and research topics.	Foreign keys to documents and research_topics; both declared cascading.
saved_research	Per-user saved document relationship and timestamp.	Foreign keys to users and documents; unique constraint on the pair; cascading deletes.
ai_questions	Persisted user question and timestamp for usage/dashboard reporting.	Optional user foreign key, set null on user deletion.
audit_logs	Activity action, user identity snapshot, resource, timestamp, details, and reviewed flag.	Optional user reference, set null on user deletion; stores username/role snapshots.

The models do not define a separate `sessions`, `embedding`, or `document_version` table. Vector records are in ChromaDB, not PostgreSQL. Uploaded binary files are filesystem objects, not database blobs.

11. API DOCUMENTATION

FastAPI registers the routes below. When the backend is running, the interactive Swagger UI is normally available at <http://localhost:8002/docs>, ReDoc at [/redoc](http://localhost:8002/redoc), and OpenAPI JSON at [/openapi.json](http://localhost:8002/openapi.json). Request/response descriptions here summarize the current source; the generated OpenAPI schema is the authoritative field-level reference.

Access labels: **Public** means no bearer token; **Authenticated** means any active account; **Researcher/Admin** means either elevated document-management role; **Admin** means administrator-only.

11.1 Authentication (/api/auth)

Method	Endpoint	Purpose / request	Response and important errors	Access
POST	/api/auth/register	Register username, email, password, optional full name. Caller-supplied role is not trusted; new role is viewer.	User response; 400 for duplicate username/email.	Public
POST	/api/auth/login	Authenticate username/email and password.	JWT bearer token and user fields; 401 for invalid credentials.	Public
GET	/api/auth/me	Return current account.	User response; 401 invalid token, 403 inactive account.	Authenticated
POST	/api/auth/change-password	Verify current password, then set new password.	Success message; 400 incorrect current password.	Authenticated
PUT	/api/auth/profile	Update username, email, and/or full name.	Updated user; 400 duplicate username/email.	Authenticated
POST	/api/auth/logout	Record logout activity; frontend discards token.	Success message.	Authenticated
POST	/api/auth/create-admin	Legacy protected default-admin creation operation.	Message; requires current admin.	Admin

11.2 Documents (/api/documents)

Method	Endpoint	Purpose / request	Response and important errors	Access
POST	/api/documents/upload/preview	Multipart file; extract metadata without creating records.	Preview metadata; 400 for unsupported/mismatched/empty/oversized file.	Researcher/Admin
POST	/api/documents/upload	Multipart file and optional title, author, year, department, type, topics, author metadata, IDs.	Document/processing result; 400 validation, 409 duplicate, 413 oversized, 500 processing/upload failure.	Researcher/Admin
GET	/api/documents/	Query skip and limit; list active documents.	Total and document array.	Authenticated
GET	/api/documents/saved/list	List saved documents for the current user.	documents array.	Authenticated
GET	/api/documents/{document_id}	Retrieve active document details and metadata.	Document fields; 404 if not found/active.	Authenticated
PUT	/api/documents/{document_id}/metadata	JSON title, author, optional publication year.	Updated metadata; 404 missing document, 422 invalid title/year.	Researcher/Admin
GET	/api/documents/{document_id}/view	Stream original file inline.	File response; 404 if record/file missing.	Authenticated
GET	/api/documents/{document_id}/download	Download original file.	File response; 404 if record/file missing.	Authenticated
DELETE	/api/documents/{document_id}	Delete document record, dependent saved/chunk records, file, and vector entries.	Success message; 403 insufficient role, 404 missing record, 500 database deletion failure.	Researcher/Admin
POST	/api/documents/{document_id}/save	Save a document for the current user.	Saved status and document ID; 404 missing/inactive document.	Authenticated
DELETE	/api/documents/{document_id}/save	Remove the current user's saved relationship.	Unsaved status and document ID.	Authenticated

11.3 Search, chat, summarization, and dashboard

Method	Endpoint	Purpose / request	Response and important errors	Access
POST	/api/search/	JSON query, optional top_k (clamped from 1 to 50).	Array of results with text, metadata, and distance; 400 empty query, 500 search failure.	Authenticated
POST	/api/chat/	JSON question, optional top_k.	Success flag, answer, and source list; 400 blank question, 500 route failure; no evidence/model errors may be returned as success: false.	Authenticated
POST	/api/summarize/	JSON document_id, optional max_tokens, temperature.	Success flag and structured/raw summary; processing/model errors may be represented as success: false; 500 unexpected failure.	Authenticated
GET	/api/summarize/document/{document_id}	Check document existence, processed flag, chunk count, and can_summarize.	Status fields; 404 missing document.	Authenticated
GET	/api/dashboard/summary	Read current user's persisted dashboard statistics.	Document, topic, AI-question, category/type, and activity data.	Authenticated

11.4 Administration (/api/admin)

Method	Endpoint	Purpose / request	Response and important errors	Access
POST	/api/admin/users	Create a user from username/email/password/full name/role.	User record; 400 duplicate username/email; 201 on success.	Admin
GET	/api/admin/dashboard	Query days (1–3650; default 30); aggregate system statistics.	Counts, roles, document classifications/status, recent users/documents, audit activity.	Admin
GET	/api/admin/activity	Optional date, start_date, end_date, limit (1–1000).	Activity history; this request marks unreviewed audit rows reviewed before response.	Admin
GET	/api/admin/activity/notifications	List recent unreviewed activity and total count.	Notification count and items.	Admin
PATCH	/api/admin/activity/{activity_id}/review	Mark one activity reviewed.	Reviewed status; 404 unknown activity.	Admin
GET	/api/admin/activity/export-report	Optional date/range filters; generate PDF from database statistics and audit rows.	application/pdf attachment; operational failures depend on database/report generation.	Admin
GET	/api/admin/users	List user accounts.	User array.	Admin
GET	/api/admin/users/{user_id}	Retrieve a user.	User fields; 404 unknown user.	Admin
PATCH	/api/admin/users/{user_id}/status	JSON is_active.	Updated status; 404 unknown user, 400 self-disable.	Admin
PATCH	/api/admin/users/{user_id}/role	JSON role enum (admin, researcher, viewer).	Updated role; 404 unknown user, 400 self-role-change.	Admin
DELETE	/api/admin/users/{user_id}	Delete a different user account.	Success message; 404 unknown user, 400 self-delete.	Admin

11.5 Service endpoints

Method	Endpoint	Purpose / request	Response and important errors	Access
GET	/	Return API service name, running status, and version.	JSON service information.	Public
GET	/api/health	Return the backend health status object.	JSON status; the current handler does not execute a database health query.	Public

The route inventory above reflects registered source routes, not a claim that every error condition has a dedicated automated API test. Endpoint integration tests using FastAPI `TestClient` or a live test server were not found in `backend/tests/`.

12. AI AND RAG FUNCTIONALITY

12.1 Embeddings and vector retrieval

`EmbeddingService` loads SentenceTransformers `all-MiniLM-L6-v2` and normalizes generated vectors. `DocumentService` creates embeddings for document chunks and query text. ChromaDB's `PersistentClient` stores embeddings and chunk text in the `research_documents` collection under `vectorstore/`. The application does not configure a remote vector database service.

12.2 Retrieval-augmented question answering

The normal chat path uses `RAGService`: retrieve similar chunks, build context containing document/chunk identifiers and relevance, and ask `LLMService` to generate a response. The system prompt directs the model to rely only on supplied context and state when that context is insufficient. Chat response cleaning removes supported `<think>` reasoning blocks before returning content.

If initialization of semantic retrieval fails, `/api/chat/` falls back to stored PostgreSQL chunks and lexical term ranking. This is a fallback implementation, not semantic retrieval, and does not guarantee the same ranking behavior.

12.3 Local language models and summaries

`LLMService` uses Ollama's `/api/generate` endpoint with `LLM_MODEL`, defaulting to `deepseek-r1:1.5b`. `SummarizationService` separately uses `SUMMARY_LLM_MODEL`, defaulting to `qwen3.5:0.8b`. `OLLAMA_HOST` defaults to `http://localhost:11434`. The two models are configurable by environment; the embedding model is hard-coded in the embedding service constructor unless the code is changed.

The summarizer creates a structured institutional brief with sections for executive summary, objective/scope, methodology/data, findings, effects, risks, implications, conclusion, and evidence gaps. The prompt requires unsupported sections to be labeled `Not stated in the source document`. Output quality is not evaluated by an automated factuality metric in this repository.

12.4 Source flow

```
User question
-> query embedding
-> ChromaDB retrieval (or database-chunk fallback)
-> selected text and source metadata
-> Ollama RAG prompt
-> answer plus source records
```

The source list contains document IDs, chunk indexes, excerpts, relevance scores, and optional metadata/page values. Exact page availability is format-dependent, as described in Section 19.

13. DOCUMENT PROCESSING

13.1 Validation and storage

The upload processor accepts `.pdf` and `.docx` only. Maximum size is 50 MiB. It checks extension and reported MIME agreement, rejects empty files, verifies actual bytes against the size limit, and uses a SHA-256 check to reject a duplicate matching an active stored document. Storage names include a timestamp and UUID and are placed in `data/documents/`.

13.2 Extraction and normalization

- PDF text and per-page content are extracted with pypdf. pdfplumber is used as a fallback when initial text extraction is limited.
- DOCX text is extracted from paragraphs and tables with python-docx. DOCX is represented as one extraction page in the metadata structure; no stable document page numbering is derived from Word pagination.
- Cleaning normalizes whitespace while preserving line boundaries that are useful to metadata rules.
- Metadata extraction/classification uses code heuristics on file text, cover text, native properties, and filename. It returns fields such as title, author(s), year/period, author source/confidence, department/category, document type, topics, and keywords where identifiable.
- Empty/weak extraction, scanned-image OCR, and OCR installation are not handled by an OCR subsystem; OCR is **Not implemented / Not verified in the current implementation**.

13.3 Chunking, embedding, and persistence

Text is sentence-split into chunks with a configured maximum target of 1,000 characters and 200-character overlap. For PDF, each page is chunked independently and each resulting chunk retains a page number. Chunk text, index, length, and optional page number are persisted in `document_chunks`. Embeddings and vector metadata are stored separately in ChromaDB. The document row stores processing state, chunk count, file reference, and extracted metadata.

Upload processing and vector indexing occur synchronously in the request path. A background job queue, retry worker, OCR pipeline, and independent processing service are **Not implemented / Not verified in the current implementation**.

14. SECURITY AND ACCESS CONTROL

Implemented controls

- JWT bearer authentication is used on protected API routes. Tokens are signed with HS256 and expire after 24 hours in code.
- Passwords are hashed through Passlib bcrypt with 12 rounds. Password verification and current-password checks exist.
- Role dependencies distinguish admin, researcher, and viewer access. Public registration always assigns viewer rather than honoring a requested elevated role.
- Protected document/AI/search/dashboard operations require an active user. Upload, document metadata edit, and document deletion require researcher or admin. Administrative user and audit operations require admin.
- Upload validation checks extension, MIME/extension agreement, actual bytes, empty content, file size, and duplicate file hashes; storage uses a UUID-generated filename.
- Selected actions are written to the database audit log. Review changes the `reviewed` flag; it does not remove the audit record.
- Pydantic request models validate many JSON bodies; backend code validates upload fields and selected metadata constraints.
- CORS allows a fixed local development origin list in `backend/app/main.py`.

Security limitations and deployment considerations

The code includes a development JWT secret fallback; deployers must supply a strong secret. The checked-in Compose database account values and admin helper's development credentials are not suitable as production secrets. The `.gitignore` file is empty. CORS origins are local development origins rather than a deployment-specific allow-list. No rate limiting, MFA, refresh token, external identity provider, password-reset workflow, malware scanning, or formal security assessment is verified in the repository. These should not be assumed to exist.

The frontend keeps the bearer access token in browser local storage. Original document view/download endpoints require authentication, but broader data-classification, retention, backup, and network policies are **Not implemented / Not verified in the current implementation**.

15. ADMINISTRATION AND AUDIT

15.1 User administration

Admin endpoints support account creation, listing, individual lookup, role update, active/inactive status changes, and account deletion. The frontend's User Management screen implements creation, role/status controls, and delete action. Self role changes, self-disable, and self-deletion are explicitly rejected for the corresponding operations.

15.2 Dashboards and monitoring

The standard dashboard reports database-backed values for the current user and active documents. The admin dashboard aggregates registered/active user counts by role; active, processed, pending, and failed document counts; topics, types, categories, questions, saved research, recent users/documents, and activity over a requested period.

The Document Processing screen groups current document rows by **pending**, **processing**, **completed**, and **failed**. Reference screens for categories, document types, and research topics display dashboard-backed records; the current frontend contains no create/edit/delete controls or matching reference-data CRUD API routes.

15.3 Audit history, notifications, and PDF reporting

`AuditLog` stores action, username/role snapshot, resource, details, date/time, timestamp, optional user foreign key, and reviewed status. The audit helper uses the Africa/Dar_es_Salaam timezone with a UTC+3 fallback. Auth, uploads, document view/download/delete, search, successful chat, successful summarization, profile/password changes, and admin actions write activity records at the relevant call sites.

Admin activity routes support date/range filters, notification counts, per-item review, and downloadable PDF export. The export report is dynamically built with ReportLab from current database totals and the selected audit records; it is not a static report file. One implementation detail to note: requesting `/api/admin/activity` marks all currently unreviewed audit records reviewed before it returns rows, while `/api/admin/activity/notifications` is a separate read of unreviewed records.

16. USER INTERFACE

Screen	Purpose and implementation status
Login	Implemented in <code>LoginPage.jsx</code> ; authenticates through the login API and passes the authenticated user to the app.
Dashboard	Implemented using backend-backed dashboard data and recent documents/topics.
Research Documents	Lists active documents with document details, open/view, download, save, Ask AI, summary, and researcher/admin delete actions.
Document Details	Displays metadata, processing state, file information, and research topics; authorized document managers can edit title, author, and year.
Semantic Search	Submits queries, displays retrieved excerpts and metadata, and offers a document-view action; page labels appear when supplied.
Ask AI / Chat	Submits a question, displays response and source records, with document/page viewing actions where source data allows.
Research Summaries	Selects a processed document, requests a summary, and displays structured/raw summary content.
Upload Document	Selects PDF/DOCX, previews and edits title/author/year metadata, asks for confirmation, then uploads and processes the document.
Saved Research	Lists the authenticated user's saved documents and allows viewing, opening, unsaving, chat, and summary actions.
Profile	Displays and edits profile fields backed by the profile API.
Settings	A screen/navigation destination exists, but functional settings persistence is not implemented/verified.
Admin Dashboard	Shows system statistics, classification charts, recent documents/users, and processing state from admin APIs.
User Management	Admin-only account creation, role/status controls, account listing, and deletion.
Document Processing	Admin monitoring view grouped by processing status.
Research Topics / Categories / Types	Admin navigation and data-backed list views exist; reference-data CRUD is not implemented in current routes/UI.
Activity / Audit Logs	Admin-only activity table, notifications, review actions, date selection, and PDF export.

The frontend is a state-driven single-page application; page selection is primarily managed in `App.jsx`. A separately verified router-based deep-link system is not used for the listed views.

17. TESTING AND VALIDATION

17.1 Current validation performed for this report

Check	Result	Scope / caveat
Backend pytest suite	Passed: 26 tests	Run on 5 October 2026 using backend/venv; tests cover direct service/model/API-function behavior, metadata extraction, auth/dashboard/audit helpers, summarization response cleaning, and recent metadata/delete persistence.
Frontend Vite production build	Passed	npm run build from frontend/; confirms bundle compilation, not browser workflow acceptance.
Frontend ESLint	Passed after cleanup	npm run lint completed without errors after removing unused state, stabilizing notification loaders, and deleting an unused chart prop.
Docker Compose startup	Not run for this report	No current container startup result is claimed.
Live PostgreSQL/Ollama integration	Not run for this report	No current server/API/AI integration result is claimed.
Browser-based end-to-end testing	Not run for this report	No browser acceptance or UI automation result is claimed.

Current test command, run from backend/:

```
.\venv\Scripts\python.exe -m pytest tests -q
```

Current frontend validation commands, run from frontend/:

```
npm run build
npm run lint
```

17.2 Existing automated test coverage

The repository's backend tests cover:

- PDF/DOCX metadata extraction rules, authors, titles, publication year, taxonomy, and file validation.
- Password hashing, invalid JWT handling, default admin creation, dashboard aggregation, user-scoped question counts, and activity review state.
- Summary response cleaning and use of a dedicated summary model.
- Persistence of manually entered author metadata and document deletion with dependent chunks/saved research.

The integration/, security/, and unit/ test directories currently contain no test modules. No REST-level TestClient suite or frontend component/browser test suite was found. The current 26 passing pytest tests therefore do not establish complete end-to-end, live database, live Ollama, performance, or AI factuality validation.

17.3 Historical validation

AUDIT_HARDENING_REPORT.md, dated 5 September 2026, records earlier compile checks, focused processor tests, live authenticated API checks, an upload smoke check, search/RAG/summary checks, and a seven-test suite at that time. It also says Docker Compose could not be executed on that workstation. Those are historical assertions in the existing report; they are not rerun results for this document. The current suite contains 26 tests and was run separately as noted above.

18. CURRENT PROJECT STATUS

No project schedule, approved completion percentage, release sign-off, or user-acceptance record is present. An 80–90% completion figure is **Not verified in the current implementation** and is not assigned here.

COMPLETED / IMPLEMENTED

- React/Vite frontend and FastAPI backend are present and build/run commands are defined.

- PostgreSQL SQLAlchemy models, startup schema creation, and initialization/seed script exist.
- Bearer-token authentication, bcrypt password hashing, and admin/researcher/viewer role checks exist.
- PDF/DOCX upload, validation, metadata extraction/review, local file storage, processing status, chunk persistence, embeddings, and ChromaDB indexing exist.
- Semantic search, database-backed lexical fallback, grounded RAG/chat, source records, and structured summaries exist.
- Document listing/details/view/download/delete and saved research routes exist.
- Admin user controls, dashboards, activity notifications/review, and PDF report export exist.
- Current backend test suite passes 26 tests; current frontend production build passes.

IN PROGRESS

A repository-backed milestone tracker or work-in-progress list was not found. Specific tasks currently in progress are **Not verified in the current implementation**.

REMAINING WORK / NOT VERIFIED

- Resolve the current frontend lint failures and hook warning.
- Add REST/API, frontend, browser, and live integration coverage beyond current direct tests.
- Establish a versioned database migration workflow; no migration tool or revisions are configured.
- Complete metadata correction for every editable classification field and verify correction behavior on existing data.
- Ensure exact page references across supported formats; DOCX pagination is not currently mapped.
- Validate Ollama model availability, AI output quality, and performance under deployment workloads.
- Prepare and test production secret handling, CORS, database credentials, backup/restore, TLS/reverse proxy, and operational monitoring.
- Complete user acceptance and deployment sign-off; these are **Not verified in the current implementation**.

19. BOT EXPERT FEEDBACK

The three feedback items below were included in the user-supplied report prompt. A dated meeting note, reviewer record, or formal feedback artifact was not found in the repository, so attribution and date are **Not verified in the current implementation**. The status column is verified against current code.

Feedback supplied for this report	Current implementation status	Required improvement / evidence
After semantic search, users should be able to view/open a returned document.	Implemented. Search results expose a view action. The frontend calls the authenticated document-view route with the result's document ID and optional page number.	Validate the workflow in browser acceptance tests, including results with missing IDs and non-PDF documents.
Source references should identify the exact source page rather than only generic chunks.	Partially implemented. PDF extraction creates page-aware chunks, database chunk rows hold <code>page_number</code> , and search/chat response schemas can return it. DOCX has no mapped pagination; source records also retain chunk identifiers.	Test exact citation behavior on PDFs; decide and implement a stable DOCX page/section citation strategy if required. Keep chunk index as technical metadata, not as a replacement for page references.
Users should review and correct metadata when automated metadata is inaccurate.	Partially implemented. Upload preview allows confirmation/editing of title, author, and year. A post-upload editor persists title, author, and year. The post-upload editor does not edit department/category, document type, topics, keywords, or all confidence/provenance fields.	Extend and validate correction of the remaining supported metadata fields; persist review provenance consistently and test manual corrections end to end.

The metadata extractor is implemented as deterministic code heuristics over the file, cover text, native properties, and filename. It is not an Ollama-generated metadata workflow in the inspected processor. The possibility of extraction mistakes is real; the report does not describe those metadata values as AI-generated unless a separate verified pipeline is added.

20. CHALLENGES AND LIMITATIONS

20.1 Document processing and metadata

Processing and embedding run synchronously inside the upload request. Long documents or model initialization can increase request latency; the repository does not configure a background worker, retry queue, or processing timeout policy. Scanned-image OCR is not present. Metadata classification is heuristic and can be incomplete or incorrect; post-upload correction is currently limited to title, author, and year.

20.2 Search, citations, and RAG

Semantic relevance thresholds and retrieval quality are not calibrated by a benchmark in the repository. The lexical database fallback is useful for availability but is not semantically equivalent to vector retrieval. PDF chunks preserve page numbers, while DOCX pagination is not mapped. RAG prompts request grounded answers, but no automated factuality/grounding score or hallucination benchmark is provided.

20.3 Model and deployment resources

Ollama models and SentenceTransformers require model files and local resources. The repository specifies no numeric hardware minimums, deployment concurrency target, latency target, or model cache policy. CPU/GPU throughput and storage requirements are therefore **Not verified in the current implementation**.

20.4 Schema and operations

SQLAlchemy `create_all()` and additive startup `ALTER TABLE` operations exist, but no versioned migration revisions or formal rollback process are present. The repository also does not provide a production backup/restore plan, TLS/reverse-proxy deployment, or environment-specific CORS configuration. The root `.gitignore` is empty, making secret and generated-file hygiene an operational concern.

20.5 Documentation drift

`PROJECT_HANDOFF.md` contains statements that no longer match the current code, including claims that login UI, saved research, document download, and document delete are absent. Current source has these features. Conversely, the existence of source code does not establish deployment readiness or institutional acceptance. For current behavior, use code and tests; update the handoff document to remove stale statements.

21. WAY FORWARD

21.1 Immediate priorities

1. Keep frontend lint clean by rerunning `npm run lint` after UI changes.
2. Add API-level tests for authentication and authorization, document upload/edit/delete, search/chat/summarization responses, and admin routes using isolated databases and mocked external services.
3. Add browser acceptance coverage for role-specific navigation, metadata editing, search-to-view, page citations, upload, saved research, and admin reporting.
4. Expand post-upload metadata correction to every field that the product requires users to review, and record provenance/review status consistently.
5. Add citation tests that verify returned PDF page numbers and document the limitation for DOCX pagination.
6. Introduce a versioned migration process before schema evolution; verify startup compatibility against clean and existing databases.
7. Update `PROJECT_HANDOFF.md` to match the current login, saved research, document download/delete, metadata editing, and test state.

21.2 Deployment and longer-term validation

1. Define production secrets and bootstrap-admin handling, database credentials, CORS origins, network exposure, backup/restore, TLS, retention, and monitoring in an approved deployment configuration.

2. Execute and record Docker Compose startup on a Docker-enabled host; validate the PostgreSQL, Ollama, backend, and frontend health paths.
3. Validate configured Ollama and embedding model downloads, resource use, latency, failure handling, and AI output quality with representative documents and questions.
4. Run stakeholder/user acceptance testing and record approved feedback, acceptance criteria, and release sign-off.
5. Establish performance targets and measure them under an agreed workload before making performance claims.

These are recommendations derived from repository gaps and known limitations. They are not claims that the work is already scheduled or approved.

22. CONCLUSION

The AI Research Knowledge Hub implements a connected document and research workflow: authenticated users can access stored documents; researchers/admins can upload, process, and correct selected metadata; users can search documents, ask questions grounded in retrieved content, request structured summaries, and save research; administrators can manage accounts, monitor activity, review audit entries, and export a PDF report.

The implementation is supported by a 26-test backend suite and successful current frontend production build and lint. Complete live service, end-to-end browser, performance, production deployment, and user-acceptance validation have not been established by this report. Important limitations include synchronous processing, incomplete post-upload metadata correction, format-dependent exact page citations, lack of a configured migration workflow, and production configuration gaps.

The system offers a practical foundation for institutional research discovery and review. Its expected value is improved access to document content and assisted review, but measurable impact and institutional acceptance remain to be established through user testing and operational validation.

23. APPENDICES

Only appendices supported by repository artifacts are included. No standalone system architecture, system design, or database diagram, formal API specification document, or UI screenshot set was found. API details are documented in Section 11; architecture and data design are described in Sections 8 and 10.

Appendix E - Installation Guide

The companion repository-verified installation guide is available as `docs/INSTALLATION_GUIDE.md` and `docs/INSTALLATION_GUIDE.pdf`. It covers prerequisites, local/Docker setup, environment configuration, initialization, verification, troubleshooting, and production notes.

Appendix G - Testing Evidence

Current test files in `backend/tests/`:

- `test_document_delete.py`
- `test_document_metadata.py`
- `test_document_processor.py`
- `test_metadata_extraction.py`
- `test_security_and_dashboard.py`
- `test_summarization_service.py`

Run from `backend/` with:

```
.\venv\Scripts\python.exe -m pytest tests -q
```

The verified run used the workspace's `backend/venv` environment and returned **26 passed**.